Tópicos Especiais de Segurança

Lucas Panonko Assis Belchior

Matrícula: 12311BCC068

Wasabi: clusterização H1

Análise de endereços de entrada e grafos de pagamentos

Repositório do projeto

https://github.com/LuksbellN/Mixer-Wasabi---H1-e-Grafo

11 de novembro de 2026

Visão geral do trabalho

01 / OBJETIVO E METODOLOGIA

Aplicar a heurística H1 aos históricos dos endereços de **entrada** das transações fornecidas, retirando os mixers das uniões. Em seguida, comparar dois grafos: um com endereços nos vértices e outro com os clusters encontrados pelo H1.

Transações do recorte	Históricos completos	Transações distintas
10	129 / 129	74

Dados e recorte

O arquivo fornecido contém **1.001 transações** e 73.454 endereços de entrada distintos. Foram selecionadas as dez primeiras transações na ordem por horário e hash, de 08/11/2017 a 15/09/2018 (UTC): 134 entradas e 129 endereços distintos. Foi usado esse recorte para limitar o número de consultas, considerando a possibilidade de divisão do trabalho mencionada pelo professor. As demais transações não foram analisadas.

Coleta e conferência dos históricos

O endpoint POST do professor respondeu HTTP 503 no teste inicial. A coleta foi feita por `blockchain.info/rawaddr`, também do Blockchain.com. O código pede 50 transações por página, aumenta o offset e espera 10 segundos entre consultas. Os JSONs são salvos para não baixar novamente o que já foi obtido. O fim da coleta é conferido pelo tamanho da última página e pelo total informado pela API. Páginas repetidas ou contagens diferentes geram erro.

Regra H1 e transitividade

H1 considera que os endereços usados como entradas de uma mesma transação são controlados pela mesma entidade [1]. Depois de remover cópias da mesma transação, identificadas pelo hash, o algoritmo segue estes passos:
1. Excluir das uniões todos os **1.001 hashes de mixer** do arquivo original.
2. Unir os endereços de entrada de cada transação restante usando union-find, uma estrutura de conjuntos disjuntos.
3. Aplicar a transitividade: se A aparece com B e B aparece com C, os três ficam no mesmo cluster. Endereços sem união permanecem separados. Não foi usada heurística de troco.

Resultado da clusterização

Foram salvas 129 páginas. Nos históricos, 10 mixers conhecidos foram excluídos das uniões; 37 transações elegíveis com múltiplos endereços geraram 263 uniões novas. Entre os 129 endereços usados na coleta, foram encontrados 128 clusters. A lista de mixers usada é a do arquivo do exercício; não foi criado um detector de CoinJoin.

Grafos antes e depois do H1

02 / GRAFOS

Mesmo conjunto de transações, duas representações

No grafo original, cada vértice é um endereço. Para cada transação, cria-se uma aresta dirigida de cada endereço distinto de entrada para cada endereço distinto de saída. No grafo agrupado, cada endereço é substituído pelo seu cluster H1. As transações de mixer permanecem como relações no grafo; sua exclusão vale para as uniões H1.

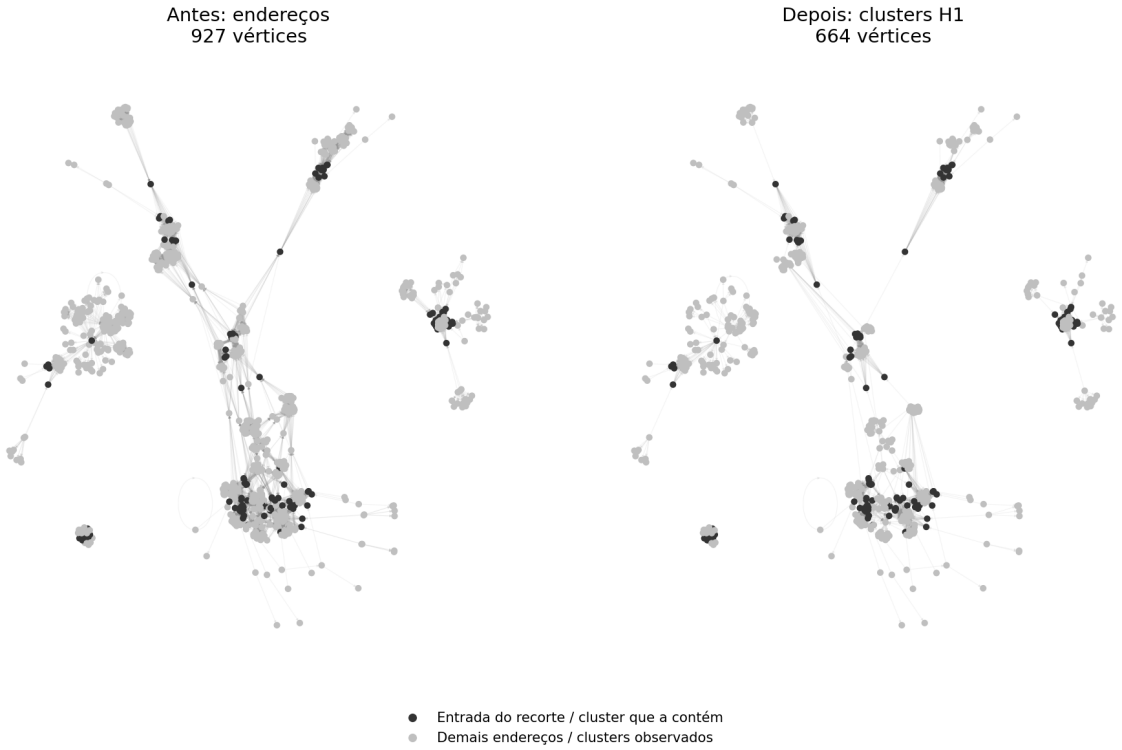


Figura 1. Os dois grafos usam as mesmas transações. Arestas repetidas entre o mesmo par aparecem sobrepostas no desenho; os arquivos GraphML e CSV guardam cada relação e o hash da transação. A posição dos vértices serve apenas para organizar a figura.

Medida	Endereços	Clusters H1
Vértices	927	664
Relações dirigidas, com multiplicidade	7.349	7.349
Laços após contração	-	39

Como ler as arestas: uma ligação indica que os endereços participaram da mesma transação como entrada e saída. Ela não informa qual entrada financiou cada saída; por isso, não recebe um valor em BTC. “Antes/depois” compara os grafos por endereço e por cluster. Isso não permite identificar qual saída do mixer pertence a cada participante.

Exemplo e análise dos resultados

03 / RESULTADOS

Uma transação usada no H1

Exemplo de 06/11/2017 (UTC): hash completo abaixo. Possui 4 endereços de entrada distintos e 2 de saída. Ela não está na lista de mixers. Seus endereços de entrada são unidos pelo H1.

```
b91f92637361407588596f6f158b684e
14554942e03b1a5339283c0a6698af60
```

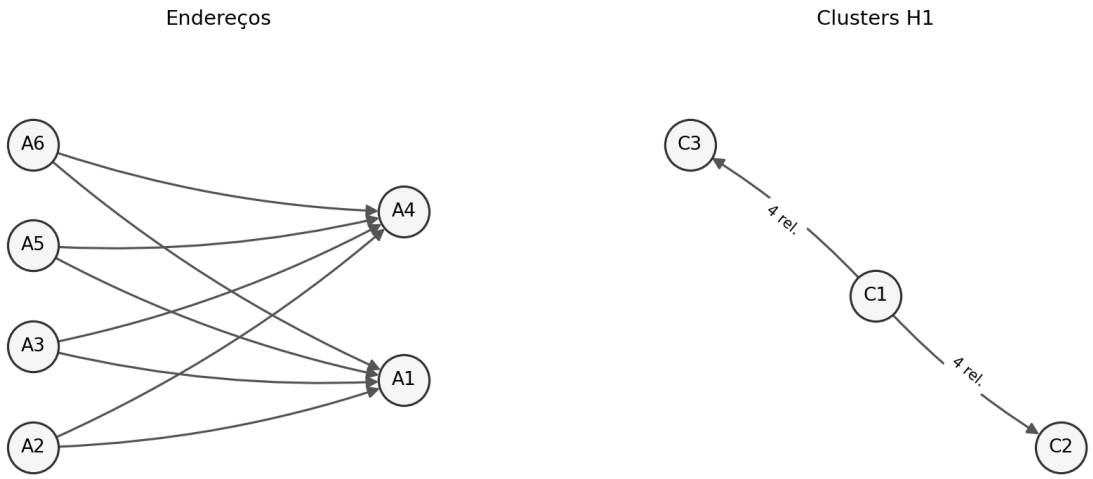


Figura 2. A mesma transação em detalhe: 6 vértices por endereço e 3 por cluster. As 8 relações originais continuam no arquivo agrupado. A marca “rel.” indica multiplicidade, sem valor monetário.

Vértice	Endereço abreviado	Cluster
A1	14CSwbAf3b7D...qJ1nzk	C3
A2	14UT6Sc7k8J2...cGvgXq	C1
A3	1A21n6ytujSV...FPbyPf	C1
A4	1BWWHNfC9659...ocqRPU	C2
A5	1BeZJkAFJJor...1gv6LX	C1
A6	1HWaWvrcotbd...GK9DgV	C1

Dos 129 endereços de entrada selecionados, 2 foram agrupados com outro endereço da seleção; os outros 127 não foram unidos a outro endereço desse conjunto. Assim, as entradas ficaram distribuídas em 128 clusters. A redução de 927 para 664 vértices da página anterior inclui também outros endereços encontrados nos históricos. Ela não deve ser confundida com a redução entre os endereços selecionados.

Os endereços completos da tabela estão em `results/example.json`. As transações usadas nas uniões estão em `h1_evidence.csv`; os JSONs baixados permitem conferir suas entradas.

Limitações e reprodução

04 / DISCUSSÃO E REPRODUÇÃO

Limitações

As dez transações foram escolhidas pela ordem de data, sem sorteio. Os resultados valem para esse recorte e não descrevem todo o uso do Wasabi.

Os históricos foram consultados em 11/11/2026. Eles podem conter transações anteriores e posteriores aos mixers; os clusters não mostram apenas o que era conhecido antes da mistura.

H1 é uma heurística, não uma identificação de pessoas. CoinJoin e PayJoin podem reunir entradas de participantes diferentes [2]. Como o filtro usa apenas a lista fornecida, outros casos desse tipo podem gerar uniões incorretas.

Entradas sem endereço: 0; saídas sem endereço: 0. Foram usados os históricos retornados pelo Blockchain.com, sem conferência em um nó Bitcoin próprio.

Como reproduzir

Python 3.12+; executar na pasta do trabalho. Os JSONs incluídos permitem refazer a análise sem novas consultas. Para refazer também a seleção, colocar o `wasabiTxs.tgz` original da disciplina na pasta superior.

```
python -m pip install -r requirements.txt

python -m unittest -v

python analyze.py prepare --count 10

python collect.py

python analyze.py run

python build_report.py

python verify.py
```

Arquivo original: SHA-256

```
5e5d695e8d15ad4a8edccfbf9ed6373d
484f5ed6ceb9e0110ac30d284695b422
```

Referências

[1] Meiklejohn et al. A Fistful of Bitcoins: Characterizing Payments Among Men with No Names. IMC, 2013. DOI: 10.1145/2504730.2504747. Artigo original.

[2] Dorier. BIP 78: A Simple Payjoin Proposal, seção Motivation. bitcoin.org/bip/78.

[3] Enunciado, orientações e arquivo fornecidos pelo professor; históricos públicos obtidos em blockchain.info/rawaddr.